Ghid de deployment și specificații tehnice

Acest document descrie arhitectura platformei de știri, stack-ul tehnologic, cerințele de rulare, configurarea, baza de date și procedurile complete de instalare — atât pentru dezvoltare locală, cât și pentru producție.

> Notă privind identificatorii: comenzile, numele de pachete (`@civicpress/backend` etc.) și structura din exemple sunt cele reale din proiect, ca să poată fi copiate exact. Înlocuiește domeniul și celelalte valori cu ale tale.
>
> Notă privind secretele: acest document **nu** conține valori reale de secrete, parole sau credențiale. Toate variabilele de mediu se configurează pornind de la `.env.example` (vezi secțiunea 6). Nu comita niciodată fișiere `.env` completate în repository.

Cuprins

1. Arhitectura pe scurt
2. Stack tehnologic
3. Cerințe de sistem
4. Structura monorepo-ului
5. Harta porturilor
6. Variabile de mediu
7. Baza de date: migrări
8. Rulare în dezvoltare
9. Build și rulare în producție (PM2)
10. Nginx (reverse proxy) și TLS
11. DNS dinamic (DDNS watchdog)
12. Media (stocare imagini/video)
13. Model de securitate și autentificare
14. Referință API
15. Model de date (Prisma)
16. Checklist de go-live în producție
17. Depanare (troubleshooting)

---

1. Arhitectura pe scurt

Aplicația este un monorepo cu trei aplicații și un pachet partajat:

- **Backend** — API REST (NestJS + Fastify + Prisma), pe portul **3000**.
- **Public** — site-ul public de știri (Angular 18 + AnalogJS, cu SSR), pe portul **4200**.
- **Admin** — panoul de administrare (Angular 18 SPA, servit sub `/admin/`), pe portul **4300**.
- **Shared** — scheme Zod și tipuri TypeScript partajate.

În producție, un **nginx** face terminarea TLS și rutează traficul către cele trei procese pornite cu **PM2**. Baza de date este PostgreSQL. Un watchdog systemd întreține un record DNS dinamic (util pentru găzduire pe conexiune rezidențială).

```
Internet
   │  HTTPS (443)
   ▼
nginx  (TLS + reverse proxy)
   │
   ├─ /        ──►  public   (4200)
   ├─ /admin/  ──►  admin    (4300)
   └─ /v1/     ──►  backend  (3000)
                        │
                        ▼
                  PostgreSQL (5432)
```

---

2. Stack tehnologic

Nivel	Tehnologie
Site public	Angular 18.2 + AnalogJS 1.22 (Vite 6 + SSR/Nitro), Tailwind CSS 3.4
Panou admin	Angular 18.2 standalone + Vite 6, state pe semnale, TipTap 3.x, Tailwind CSS 3.4
Backend	NestJS 10, Fastify 4, Prisma 5.14, PostgreSQL 16
Autentificare	Hash parole Argon2id, JWT în cookie-uri httpOnly, RolesGuard
Partajat	Scheme Zod, TypeScript 5.5.4
Media	Sharp (strip EXIF + redimensionare la cerere), cache pe disc
Audit	Interceptor care loghează operațiile mutante în tabela `AuditLog`
Notificări	Email prin Resend, Signal prin signal-cli-rest-api
Monorepo	pnpm workspaces (pnpm 9)
Proces manager	PM2 (producție)
Reverse proxy	nginx + certbot (Let's Encrypt)
DNS dinamic	script bash + systemd timer + cPanel UAPI

---

3. Cerințe de sistem

Cerință	Versiune / detalii
Node.js	`>= 20`
pnpm	`>= 9` (fixat: `pnpm@9.15.1`; se poate activa cu `corepack enable`)
PostgreSQL	16 (în dev poate fi pornit prin Docker; credențialele se configurează în `.env`)
Docker	Opțional, pentru a rula Postgres în dev
PM2	Pentru rularea în producție
nginx	Reverse proxy + terminare TLS
certbot	Emitere certificate TLS
systemd	Pentru watchdog-ul DDNS

Nu există fișier `.nvmrc`; se folosește Node 20+.

---

4. Structura monorepo-ului

```
apps/
  backend/    — NestJS + Fastify + Prisma (API REST)
    prisma/
      schema.prisma        — modelul de date
      migrations/          — 21 migrări
    src/modules/           — auth, users, posts, media, audit, categories,
                             layout, settings, exchange-rates, notifications,
                             pills, health
    storage/media/         — fișiere media (orig + cache) la runtime
  public/     — Angular 18 + AnalogJS (SSR) — site public
    src/app/pages/         — rutare pe fișiere (home, articol, categorie, video,
                             contact, cookies, pp, tc)
  admin/      — Angular 18 SPA (servit sub /admin/)
    src/app/pages/         — login, dashboard, posts, categories, pills,
                             layouts, settings, notifications, users, audit
packages/
  shared/     — scheme Zod + tipuri TS (Role, PostStatus, PostCreate, …)
scripts/
  ops/                   — scripturi DDNS watchdog (systemd)
start.sh                 — lansator stack de dezvoltare (o comandă)
start-services-pm2.sh    — build + pornire producție cu PM2
deploy-nginx-civic.sh    — instalare config nginx
nginx.civic.probabilistik.dev.conf — config nginx sursă
docker-compose.yml       — Postgres + Adminer (dev)
```

---

5. Harta porturilor

Port	Serviciu	Bind (producție)	Note
3000	API backend (NestJS)	`0.0.0.0`; nginx → `127.0.0.1:3000`	variabila `PORT`
4200	Site public (AnalogJS SSR / Vite)	`127.0.0.1` (via `NITRO_HOST`)	
4300	Admin SPA (Vite preview)	`127.0.0.1`	`base: '/admin/'`
5432	PostgreSQL	`localhost` (Docker în dev)	
8080	Adminer (UI DB, doar dev)	`localhost`	atenție: coincide cu `SIGNAL_API_URL` implicit
80 / 443	nginx	public	HTTP → redirect HTTPS

Endpoint de sănătate: `GET http://localhost:3000/v1/health` → `{ ok, ts, db }`.

---

6. Variabile de mediu

Sursa unică de adevăr este fișierul `.env.example` din rădăcina proiectului. Toate valorile (inclusiv secretele) se completează local, nu în acest document. Copiază șablonul în fișierele `.env` ale aplicațiilor:

```
cp .env.example apps/backend/.env
cp apps/backend/.env apps/public/.env
cp apps/backend/.env apps/admin/.env
```

> Fișierele `.env` conțin secrete și **nu** trebuie comise în repository. Generează secrete puternice și proprii (de ex. `openssl rand -hex 32` pentru cheile de semnare) direct pe serverul de producție.

Mai jos este descris **rolul** fiecărei variabile, fără valori sensibile — pentru valorile concrete/implicite consultă `.env.example`.

6.1 Backend (`apps/backend/.env`)

Variabilă	Rol
`DATABASE_URL`	Conexiunea PostgreSQL
`PORT`	Portul API (implicit 3000)
`JWT_SECRET`	Cheie de semnare JWT — **obligatoriu propriu și puternic în producție**
`JWT_ACCESS_TTL`	Durata token-ului de acces (secunde)
`JWT_REFRESH_TTL`	Durata token-ului de refresh (secunde)
`COOKIE_SECRET`	Cheie de semnare a cookie-urilor — **propriu în producție**
`MEDIA_BASE_URL`	Prefix URL pentru fișierele media
`MEDIA_STORAGE_DIR`	Directorul de stocare media
`PUBLIC_URL`	Origine CORS + cookie-uri securizate în producție
`ADMIN_URL`	Link în mesajele de notificare
`RESEND_API_KEY`	Cheie API Resend; gol = email dezactivat
`MAIL_FROM`	Adresa expeditor pentru email
`SIGNAL_API_URL`	Endpoint signal-cli-rest-api
`SIGNAL_SENDER_NUMBER`	Număr expeditor Signal; gol = Signal dezactivat
`NODE_ENV`	`production` activează cookie-uri `secure` și logare optimizată
`LOG_LEVEL`	Nivel de logare (pino)

6.2 Public (`apps/public/.env`)

Variabilă	Rol
`VITE_API_BASE_URL`	Baza API în browser (client). În producție se setează gol (`""`) la build pentru a folosi căi relative `/v1`.
`BACKEND_URL`	Folosită de SSR (fișierele `*.server.ts`) pentru fetch pe server. **De setat explicit în producție** dacă backend-ul nu e co-localizat.

6.3 Admin (`apps/admin/.env`)

Variabilă	Rol
`VITE_API_BASE_URL`	Prezentă în șablon; codul admin folosește oricum căi relative `/v1`, servite prin nginx.

6.4 DDNS (`scripts/ops/ddns-watchdog.env`)

Se pornește de la `scripts/ops/ddns-watchdog.env.example`. Conține date sensibile (credențiale cPanel) — completează-le doar local.

Variabilă	Rol
`DOMAIN` / `DNS_HOST`	Domeniul și hostname-ul verificat
`AUTO_UPDATE`	Actualizează automat record-ul A în cPanel la drift
`ALERT_EMAIL`	Alerte email opționale
`CPANEL_HOST`	Host cPanel
`CPANEL_USER` / `CPANEL_API_TOKEN`	Credențiale API cPanel (**secrete**)
`TARGET_NAME`	Record A apex
`TTL`	TTL DNS (secunde)

---

7. Baza de date: migrări

7.1 Migrări Prisma

Migrările se află în `apps/backend/prisma/migrations/` (21 migrări; provider `postgresql`).

- **Dev (interactiv, creează migrare nouă):** `pnpm db:migrate` (rulează `prisma migrate dev`)
- **Dev automat (via `start.sh`):** `prisma migrate deploy`, dacă detectează migrări în așteptare
- **Producție:** `pnpm --filter @civicpress/backend exec prisma migrate deploy`
- **Prisma Studio (browser DB):** `pnpm db:studio` (rulează `prisma studio`)
- **Generare client Prisma:** `pnpm --filter @civicpress/backend prisma:generate`

7.2 Postgres în dezvoltare (Docker)

`docker-compose.yml` pornește Postgres 16 și Adminer (credențialele se configurează prin variabilele de mediu / `.env`):

```
pnpm db:up    # docker compose up -d postgres adminer
pnpm db:down  # docker compose down
```

Detalii container: port `5432:5432`, volum persistent `postgres_data`, healthcheck `pg_isready`. Adminer (UI de administrare a bazei de date, doar pentru dev) pe `:8080`.

---

8. Rulare în dezvoltare

8.1 Cea mai simplă cale (`start.sh`)

```
pnpm install
./start.sh            # pornește tot (backend + public + admin)
```

Ce face `start.sh`:

1. Se asigură că daemon-ul Docker rulează.

2. Verifică Postgres pe `:5432`; dacă nu răspunde, pornește `docker compose up -d postgres`.
3. Verifică migrările în așteptare; dacă există, rulează `prisma migrate deploy`.
4. Oprește procese vechi de pe porturile 3000, 4200, 4300.
5. Pornește cele trei servicii (loguri în `.logs/`):
   - Backend: `ts-node --swc src/main.ts` → `:3000`
   - Public: `vite` → `:4200`
   - Admin: `vite` → `:4300`
6. Așteaptă porturile și health check-ul backend-ului, apoi urmărește logurile. `Ctrl+C` oprește tot.

URL-uri: Public `http://localhost:4200`, Admin `http://localhost:4300`, Backend `http://localhost:3000`.

8.2 Manual (fără `start.sh`)

```
cp .env.example apps/backend/.env
cp apps/backend/.env apps/public/.env
cp apps/backend/.env apps/admin/.env

pnpm db:migrate     # rulează migrările
pnpm dev            # pornește toate aplicațiile (concurrently)
```

8.3 Comenzi utile

```
pnpm dev            # toate aplicațiile în dev
pnpm build          # build toate (shared → backend → public → admin)
pnpm lint           # lint pe toate workspace-urile
pnpm typecheck      # verificare de tipuri
pnpm verify:visual  # smoke test vizual (Playwright)
pnpm e2e            # test e2e (Playwright)
```

---

9. Build și rulare în producție (PM2)

Scriptul `start-services-pm2.sh` face build-ul complet și pornește cele trei procese cu PM2.

```
./start-services-pm2.sh
```

Preflight: necesită `pm2`, `node`, `pnpm` (activează pnpm prin corepack dacă lipsește).

Secvența de build executată de script:

```
pnpm install --frozen-lockfile || pnpm install
# forțează căi relative /v1 în client
export VITE_API_BASE_URL=""
pnpm --filter @civicpress/backend prisma:generate
pnpm --filter @civicpress/backend exec prisma migrate deploy
pnpm --filter @civicpress/shared build
pnpm --filter @civicpress/backend build
pnpm --filter @civicpress/public build
pnpm --filter @civicpress/admin build -- --base /admin/
```

Procesele PM2 pornite (3):

`civic-backend` — API backend

- Port: `3000`
- CWD: `apps/backend`
- Comandă: `node dist/apps/backend/src/main.js`
- Variabile: `PORT=3000`, `PUBLIC_URL`, `ADMIN_URL`, `MEDIA_BASE_URL` (setate către domeniul de producție)

`civic-public` — site public (SSR)

- Port: `4200`
- CWD: rădăcina proiectului
- Comandă: `node apps/public/dist/analog/server/index.mjs`
- Variabile: `PORT=4200`, `HOST=127.0.0.1`, `NITRO_PORT=4200`, `NITRO_HOST=127.0.0.1`, `NITRO_APP_BASE_URL="/"`

`civic-admin` — panou admin

- Port: `4300`
- CWD: rădăcina proiectului
- Comandă:

  ```
  pnpm --filter @civicpress/admin preview -- \
    --host 127.0.0.1 --port 4300 --strictPort
  ```

După pornire, scriptul rulează `pm2 save`.

Comenzi PM2 utile:

```
pm2 status
pm2 logs civic-backend
pm2 restart civic-public
pm2 save
# opțional: pornire automată la boot (nu e configurat de script)
pm2 startup
```

> Note importante despre căile de build:
>
> - Backend-ul se pornește din `dist/apps/backend/src/main.js` (nu `dist/main.js` cum apare în scriptul `start` al pachetului).

> - Site-ul public se pornește din `apps/public/dist/analog/server/index.mjs`.
> - Verifică efectiv aceste căi pe serverul țintă după `pnpm build`, deoarece pot diferi de exemplele din README.
> - Admin-ul este servit prin `vite preview` (nu printr-un server static dedicat).
> - `BACKEND_URL` nu este setat de scriptul PM2; SSR-ul public folosește implicit `http://localhost:3000`. Setează-l explicit dacă backend-ul nu e co-localizat.

---

10. Nginx (reverse proxy) și TLS

Config sursă: `nginx.civic.probabilistik.dev.conf`. Instalare cu `deploy-nginx-civic.sh`.

Destinații:

- Config: `/etc/nginx/sites-available/civicpress.ro`
- Symlink: `/etc/nginx/sites-enabled/civicpress.ro`
- Certificate TLS: `/etc/letsencrypt/live/civicpress.ro/{fullchain.pem,privkey.pem}`

Instalare / reîncărcare:

```
./deploy-nginx-civic.sh
sudo nginx -t
sudo systemctl reload nginx
```

Emitere certificat prima dată (când nu există încă):

```
sudo certbot --nginx -d civicpress.ro -d www.civicpress.ro
./deploy-nginx-civic.sh    # re-deploy cu config TLS
```

> Dacă nu există încă certificat, scriptul instalează o configurație temporară doar-HTTP cu aceleași reguli de rutare, apoi îți cere să rulezi certbot.

Reguli de rutare (blocul HTTPS):

Locație	Acțiune	Upstream
`/.well-known/acme-challenge/`	fișiere statice	`/var/www/letsencrypt`
`/v1/`	proxy	`http://127.0.0.1:3000/v1/`
`= /admin`	redirect 301	`/admin/`
`/admin/`	proxy (cu upgrade WebSocket)	`http://127.0.0.1:4300/admin/`
`/api/_analog/`	rewrite `^/api/(.*)$ /$1` + proxy	`http://127.0.0.1:4200`
`/`	proxy (cu upgrade WebSocket)	`http://127.0.0.1:4200`

Setări proxy (toate locațiile): `client_max_body_size 20m`; `proxy_read_timeout 120s`, `proxy_send_timeout 120s`, `proxy_connect_timeout 10s`; headere `Host`, `X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`; headere de securitate HSTS, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`.

Blocul HTTP: servește ACME challenge și redirecționează 301 către HTTPS.

---

11. DNS dinamic (DDNS watchdog)

Util pentru găzduire pe o conexiune cu IP dinamic (rezidențială). Runbook complet: `docs/deployment/dynamic-dns-runbook.md`.

Mecanism:

1. `scripts/ops/ddns-watchdog.sh` (rulat de un systemd timer la fiecare 2 minute):
   - obține IP-ul WAN: `curl -4fsS https://api.ipify.org`
   - obține record-ul A din DNS: `dig +short ${DNS_HOST} A`
   - dacă IP-ul diferă și `AUTO_UPDATE=1` → apelează `update-cpanel-root-a.sh`
2. `scripts/ops/update-cpanel-root-a.sh` — editează record-ul A apex prin cPanel UAPI Zone Editor (autentificare cu `Authorization: cpanel ${CPANEL_USER}:${CPANEL_API_TOKEN}`, TTL implicit 300).

Instalare:

```
chmod +x scripts/ops/ddns-watchdog.sh \
         scripts/ops/update-cpanel-root-a.sh

# copiază șablonul, apoi editează valorile
cp scripts/ops/ddns-watchdog.env.example \
   scripts/ops/ddns-watchdog.env

./scripts/ops/install-ddns-watchdog.sh
```

Unitatea systemd (`ddns-watchdog.timer`): `OnBootSec=1min`, `OnUnitActiveSec=2min`, `Persistent=true`.

Monitorizare:

```
systemctl status ddns-watchdog.timer
systemctl status ddns-watchdog.service
journalctl -u ddns-watchdog.service -n 100 --no-pager
```

Recomandări DNS: TTL 300 la record-ul A apex; `www` ca CNAME către hostname-ul DNS dinamic.

---

12. Media (stocare imagini/video)

- Stocare pe disc în `MEDIA_STORAGE_DIR` (implicit `apps/backend/storage/media/{orig,cache}`). Nu se servește prin nginx static, ci prin API: `GET /v1/media/:id/:variant`.
- Tipuri acceptate: JPEG, PNG, WebP (max **10 MB**); MP4 (max **100 MB**). Tipul MIME este verificat real (biblioteca `file-type`).
- Procesare imagini (Sharp): aplică rotația EXIF, elimină metadatele; la upload pre-generează variantele `thumb` (160px), `medium` (500px), `large` (1080px) în WebP.

- Variante la cerere prin URL: `45`, `64`, `90`, `160`, `240`, `272`, `500`, `660`, `768`, `1080`, `orig` — cache-uite în `storage/media/cache/`.
- Videoclipurile se servesc doar în original.
- Limita de upload multipart la backend: 10 MB (nginx permite până la 20 MB pe request).

---

13. Model de securitate și autentificare

- **Parole**: hash Argon2id.
- **JWT în cookie-uri httpOnly**: `access_token` (implicit 1h) și `refresh_token` (implicit 30 zile). Se acceptă și `Authorization: Bearer <token>`. Flag-uri cookie: `httpOnly`, `sameSite: lax`, `secure` în producție.
- **Guarduri globale**:
  - `ThrottlerGuard` — **200 requests / 60s** global (login-ul e limitat suplimentar la **5/min**).
  - `JwtAuthGuard` — toate rutele cer autentificare, cu excepția celor marcate `@Public()`.
  - `RolesGuard` — aplică `@Roles(...)` unde e cazul.
  - `AuditInterceptor` — loghează operațiile POST/PATCH/PUT/DELETE.
- **CORS**: origini permise `localhost:4200`, `localhost:4300`, `PUBLIC_URL`, `ADMIN_URL`; `credentials: true`.
- **Helmet**: activ (CSP dezactivat; `crossOriginResourcePolicy: cross-origin`).
- **De configurat obligatoriu în producție**: secrete proprii și puternice pentru `JWT_SECRET` și `COOKIE_SECRET` (nu valorile din `.env.example`); setează `NODE_ENV=production`.

Roluri: `AUTHOR`, `EDITOR`, `ADMIN`.

---

14. Referință API

Toate rutele sunt sub `http://<host>:3000` (fără prefix global `/api`; calea `v1/...` e inclusă în controllere). „AUTHOR+" = AUTHOR/EDITOR/ADMIN; „EDITOR+" = EDITOR/ADMIN.

Auth

Metodă	Cale	Acces	Descriere
POST	`/v1/auth/login`	Public (5/min)	Login; setează cookie-uri
POST	`/v1/auth/refresh`	Public (cookie refresh)	Rotește token-ul de acces
POST	`/v1/auth/logout`	Autentificat	Șterge cookie-urile
GET	`/v1/auth/me`	Autentificat	Profil minimal

Utilizatori

Metodă	Cale	Acces	Descriere
GET	`/v1/me`	Autentificat	Profil complet + preferințe
PATCH	`/v1/me`	Autentificat	Actualizează profil/preferințe/pseudonime
GET	`/v1/users`	ADMIN	Listă utilizatori
GET	`/v1/users/:id`	ADMIN	Detaliu utilizator
POST	`/v1/users`	ADMIN	Creează utilizator
PATCH	`/v1/users/:id`	ADMIN	Actualizează utilizator
POST	`/v1/users/invite`	ADMIN	Invită utilizator (alias creare)

Articole

Metodă	Cale	Acces	Descriere
GET	`/v1/home`	Public	Layout pagina principală (module + sidebar)
GET	`/v1/posts/:slug`	Public	Articol publicat (+ incrementează vizualizările)
GET	`/v1/posts`	Autentificat*	Listă admin (autorii văd doar articolele proprii)
GET	`/v1/admin/posts/:id`	Autentificat*	Articol (orice status) pentru editare
POST	`/v1/posts`	Autentificat*	Creează ciornă
PATCH	`/v1/posts/:id`	Autentificat*	Actualizează articol
POST	`/v1/posts/:id/transition`	Autentificat*	Tranziție de workflow
DELETE	`/v1/posts/:id`	Autentificat*	Șterge articol
GET	`/v1/posts/:id/revisions`	Autentificat*	Istoric versiuni

`*` restricții pe roluri aplicate la nivel de serviciu.

Media

Metodă	Cale	Acces	Descriere
GET	`/v1/media/:id`	Public	Metadate media
GET	`/v1/media/:id/:variant`	Public	Fișier (variantă)
POST	`/v1/media/upload`	Autentificat	Upload fișier
GET	`/v1/media`	Autentificat	Listă media paginată

Categorii & meniu

Metodă	Cale	Acces	Descriere
GET	`/v1/menu`	Public	Arbore de meniu
GET	`/v1/categories/:slug`	Public	Pagină categorie (+ layout pe pag. 1)
GET	`/v1/admin/categories`	AUTHOR+	Listă categorii + contoare
POST	`/v1/admin/categories`	EDITOR+	Creează
PATCH	`/v1/admin/categories/:id`	EDITOR+	Actualizează
DELETE	`/v1/admin/categories/:id`	EDITOR+	Șterge
PUT	`/v1/admin/categories/order`	EDITOR+	Reordonează meniul

Layout

Metodă	Cale	Acces	Descriere
GET	`/v1/posts-feed`	Public	Flux „load more"
GET	`/v1/admin/layout`	EDITOR+	Configurări layout (home/category/article)
PUT	`/v1/admin/layout/:page`	EDITOR+	Salvează layout
POST	`/v1/admin/layout/pin-front`	EDITOR+	Fixează articol într-un modul

Setări

Metodă	Cale	Acces	Descriere
GET	`/v1/settings/public`	Public	Setări publice (branding)
GET	`/v1/admin/settings`	EDITOR+	Setări admin
PATCH	`/v1/admin/settings`	ADMIN	Actualizează setări

Diverse

Metodă	Cale	Acces	Descriere
GET	`/v1/exchange-rates`	Public	Curs valutar BNR (cache 1h)
GET	`/v1/admin/pills`	AUTHOR+	Listă pill-uri
POST/PATCH/DELETE	`/v1/admin/pills...`	EDITOR+	Gestionare pill-uri
GET	`/v1/audit`	Autentificat**	Jurnal de audit
GET	`/v1/health`	Public	Health check (`SELECT 1`)

`**` ADMIN vede tot; ceilalți au nevoie de `?mine=1`.

Note funcționale:

- Statusul `SCHEDULED` și câmpul `scheduledFor` există în schemă, dar **nu există cron de publicare programată** care să le folosească.
- Notificările (email/Signal) se declanșează doar la `action: "submit"` pe tranziție și sunt „fire-and-forget" (eșecul lor nu afectează răspunsul HTTP).
- Cursul valutar se ia din `https://www.bnr.ro/nbrfxrates.xml`, cu cache în memorie de 1 oră.

---

15. Model de date (Prisma)

Schema: `apps/backend/prisma/schema.prisma`.

Enums:

Enum	Valori
`Role`	`AUTHOR`, `EDITOR`, `ADMIN`
`PostStatus`	`DRAFT`, `IN_REVIEW`, `SCHEDULED`, `PUBLISHED`, `ARCHIVED`

Modele principale:

SiteSettings — singleton (`id=default`)

- Câmpuri: branding, tipografie, meniu antet, HTML pagini legale, URL-uri sociale
- Relații: → `logoMedia`, `faviconMedia`

PageLayout

- Câmpuri: `id` = `home`/`category`/`article`; `config` JSON

Media

- Câmpuri: `filename`, `mimeType`, `width/height`, `sizeBytes`, `path`, `variants` JSON, `alt`, `caption`
- Relații: ← `owner` (User); → posturi, logo, avataruri

User

- Câmpuri: `email`, `passwordHash`, `displayName`, `slug`, `bio`, `role`, `isActive`, `lastLoginAt`, preferințe notificări, `pseudonyms[]`
- Relații: → avatar; → posturi (author/submitter/approver), media, revizii, audit

Category

- Câmpuri: `name`, `slug`, `color`, `description`, `parentId`, `showInMenu`, `menuOrder`, `externalId`
- Relații: auto-referențială părinte/copii; → posturi (PostCategory)

Pill

- Câmpuri: `text`, `color`, `textColor`

- Relații: → posturi

Tag

- Câmpuri: `name`, `slug`, `externalId`
- Relații: → posturi (PostTag)

Post

- Câmpuri: `title`, `slug`, `excerpt`, `contentHtml`, `heroMediaId`, `pillId`, `videoUrl`, `authorId`, `authorName`, bife de vizibilitate, `viewCount`, `status`, `scheduledFor`, câmpuri SEO, `externalId`, timestamps
- Relații: → heroMedia, pill, author/submittedBy/approvedBy; ↔ categorii, taguri, bulbi, revizii

PostCategory

- Câmpuri: PK compus `(postId, categoryId)`, `isPrimary`
- Relații: Post ↔ Category

PostTag

- Câmpuri: PK compus `(postId, tagId)`
- Relații: Post ↔ Tag

PostBulb

- Câmpuri: PK compus `(postId, linkedPostId)`, `sortOrder`, `labelOverride`
- Relații: Post → Post legat

Revision

- Câmpuri: snapshot `title`, `excerpt`, `contentHtml`, `editorId`, `message`
- Relații: → post, editor

AuditLog

- Câmpuri: `action`, `entity`, `entityId`, `diff` JSON, `ip`, `userAgent`
- Relații: → actor (User)

---

16. Checklist de go-live în producție

1. **DNS**: instalează DDNS watchdog (dacă IP-ul WAN e dinamic) — `./scripts/ops/install-ddns-watchdog.sh`. Verifică record-ul A apex.
2. **Baza de date**: asigură-te că rulează un PostgreSQL de producție. Configurează `apps/backend/.env` cu `DATABASE_URL` de producție.
3. **Secrete**: pornind de la `.env.example`, setează secrete proprii și puternice pentru `JWT_SECRET` și `COOKIE_SECRET` (ex. `openssl rand -hex 32`), `NODE_ENV=production` și valorile corecte pentru `PUBLIC_URL`, `ADMIN_URL`, `MEDIA_BASE_URL`.

4. **Notificări** (opțional): completează `RESEND_API_KEY` / `MAIL_FROM` și/sau `SIGNAL_API_URL` / `SIGNAL_SENDER_NUMBER`.
5. **Build + start**: `./start-services-pm2.sh` (face migrate deploy, build și pornește PM2).
6. **Persistență la boot** (opțional): `pm2 startup` + `pm2 save`.
7. **Nginx + TLS**: `./deploy-nginx-civic.sh`, apoi `sudo certbot --nginx -d <domeniu> -d www.<domeniu>`, apoi re-deploy.
8. **Verificare**:

```
curl -sf http://127.0.0.1:3000/v1/health
curl -I https://civicpress.ro/
curl -I https://civicpress.ro/admin/
curl -I https://civicpress.ro/v1/health
pm2 status
```

9. **Securitate**: creează un cont ADMIN real cu parolă puternică; dezactivează/șterge orice cont de test; asigură-te că niciun fișier `.env` nu a fost comis în repository.

---

17. Depanare (troubleshooting)

Simptom	Cauză probabilă / soluție
Backend-ul nu pornește	Verifică `DATABASE_URL` și că Postgres răspunde pe 5432; vezi `pm2 logs civic-backend`.
„Migrări în așteptare"	Rulează `pnpm --filter @civicpress/backend exec prisma migrate deploy`.
Site-ul public nu ajunge la API (SSR)	Setează explicit `BACKEND_URL` pentru procesul `civic-public`.
Imaginile nu se încarcă în admin	Verifică `MEDIA_BASE_URL` și permisiunile pe `apps/backend/storage/media`.
Upload respins	Fișier peste limită (10 MB imagini / 100 MB video) sau tip MIME neacceptat; nginx limitează la 20 MB pe request.
401 repetat în admin	Cookie-uri blocate; verifică `PUBLIC_URL` / `ADMIN_URL` în CORS și că ești pe HTTPS în producție (`secure`).
Cursul valutar nu apare	BNR indisponibil; serviciul returnează cache vechi sau gol (fără eroare fatală).
Emailurile/Signal nu pleacă	`RESEND_API_KEY` / `SIGNAL_SENDER_NUMBER` gol → canalul e dezactivat intenționat.
Portul 8080 ocupat	Adminer (dev) și `SIGNAL_API_URL` implicit folosesc ambele 8080 — schimbă unul.
DNS nu se actualizează	`journalctl -u ddns-watchdog.service`; verifică `CPANEL_*` și `AUTO_UPDATE=1`.
Căi de build greșite la PM2	Verifică pe server unde a emis efectiv `pnpm build` (`dist/apps/backend/src/main.js`, `apps/public/dist/analog/server/index.mjs`).